Principles of Concurrent Programming

Verifying Concurrent Programs

Lorenzo Ceragioli

IMT
SCUOLA
ALTI STUDI
LUCCA

What is System Verification?

IMT SCUOLA ALTI STUDI LUCCA

Establishing whether the system under consideration possesses certain properties

Usually: more time and effort on verification and validation than on construction

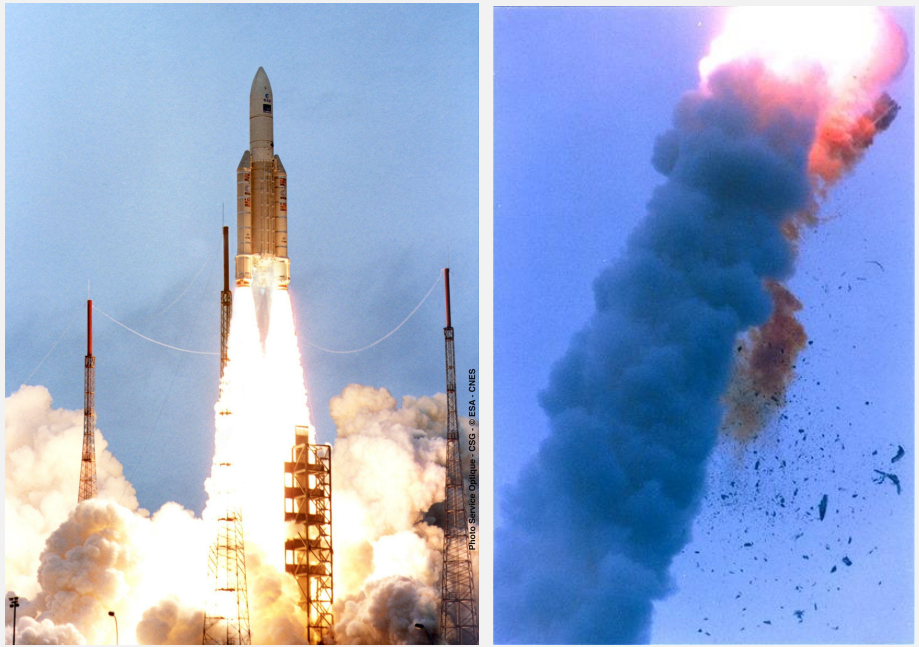
Verification = “are we building the thing **right**?”
Validation = “are we building the **right** thing?”

Note: Correctness is always relative to a specification

Because

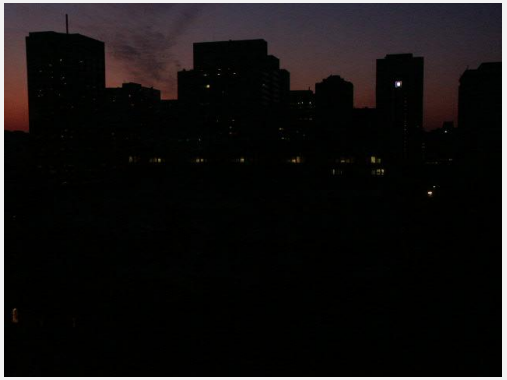
IMT SCUOLA ALTI STUDI LUCCA

- The number of defects grows exponentially with the number of interacting system components (concurrency, nondeterminism)
- Some systems cannot be (easily) fixed after release
- Failures in critical systems may be catastrophic
- In catching software errors, the sooner is the better



Explosion of first Ariane 5 flight, 1996
(overflow while converting from 64-bit floating point to 16-bit signed integer)

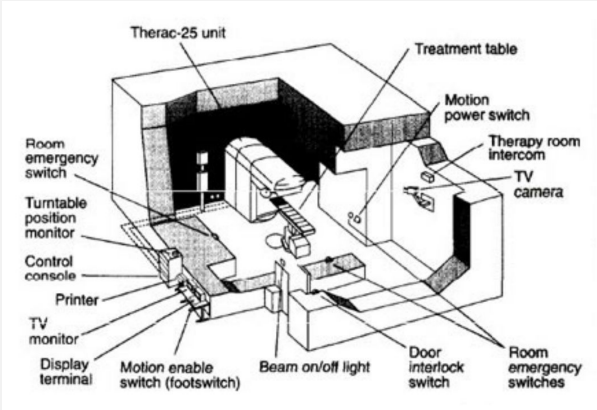
Explosion of first Ariane 5 flight, 1996
(Overflow during data conversion)



Northeast blackout, 2003
(Mishandled race condition)

IMT SCUOLA ALTI STUDI LUCCA

Pentium FDIV bug, 1994
(Missing values in a lookup table)



Therac-25 Radiation Overdosing, 1985-87
(Mishandled race condition)

DAO attack on Ethereum, 2016
(Reentrancy problem)

Informal Approaches to System Verification

IMT SCUOLA ALTI STUDI LUCCA

Software inspection carried out by a team of engineers (**peer review**)

- static technique: manual code inspection
- subtle errors are hard to catch (e.g. concurrency)

Software **simulation** (of a model) and **testing** (of a program)

- feed certain inputs, i.e., the tests
- observe reaction and check whether this is “desired”
- number of possible behaviours is very large
- unexplored behaviours may contain the fatal bug

Formal Methods

IMT SCUOLA ALTI STUDI LUCCA

Deductive Methods

Associate logical statements and derivation rules to program constructs, and derive a proof of the property for the system.

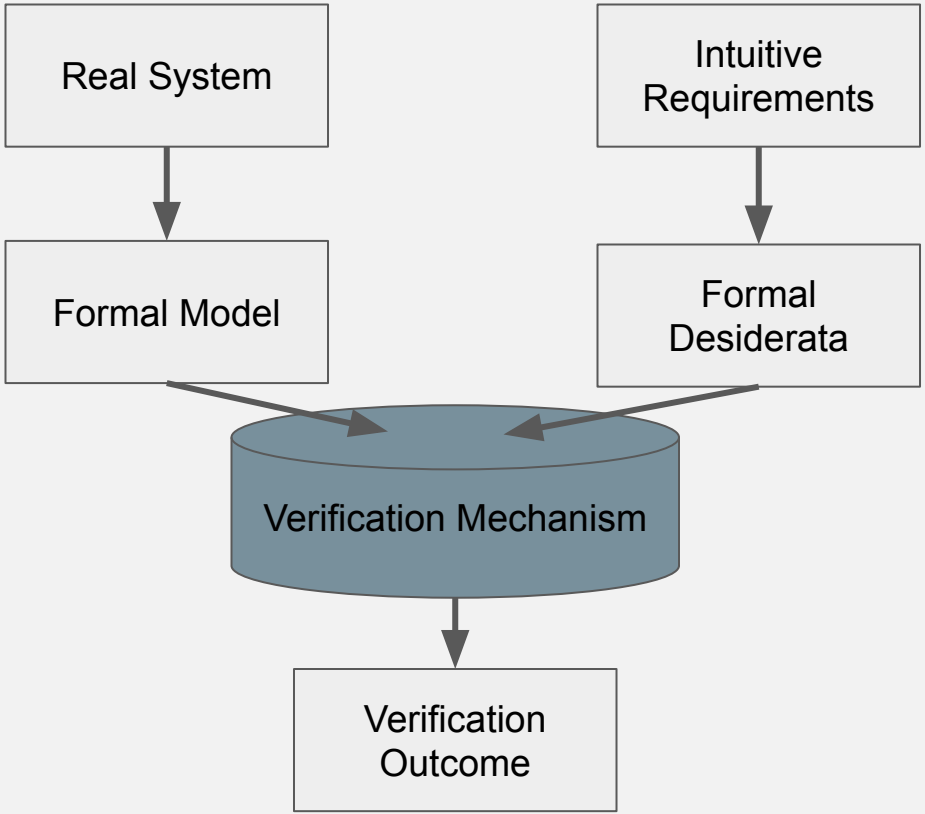
E.g. dependent types, proof assistants, Hoare logic …

Model-Based Methods

Generate and inspect a model describing the system behavior in a mathematically precise and unambiguous manner.

E.g. formal simulation and testing, **model checking** …

Objective of Verification



Interacting Systems Redefine Correctness

IMT SCUOLA ALTI STUDI LUCCA

Turing Machines

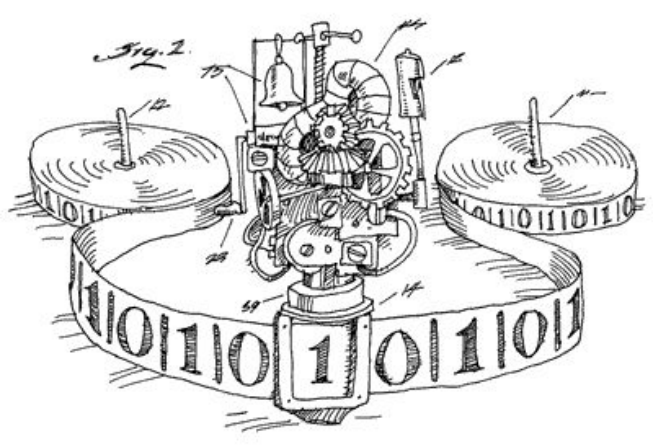
semantics = partial function on N

requirements = function equivalence

Interactive Programs: Not so easy, semantics depends on the interaction

Notice: we interact also with the Turing Machine (giving an initial state for the tape and reading the last one), but the interaction is not part of the model.

Notice: interactive does not means only with humans (see Operating Systems)



Idea: Infinite Runs have a Meaning

IMT SCUOLA ALTI STUDI LUCCA

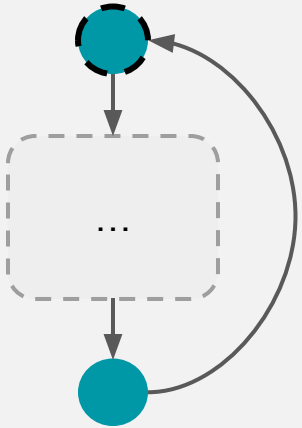
Different non-terminating behaviours may have different interaction capabilities!

Turing Machines: all the machines that never terminates have the same semantics, i.e. the (partial) function always undefined

Typical structure of a server

```
while true do {
    // receive request
    // serve request
}
```

But **not all servers are the same!**



A General Model for Concurrent Systems

IMT SCUOLA ALTI STUDI LUCCA

A *Kripke Transition System* K is a sixtuple (S, Act, $\rightarrow$, I, AP, L), where

- S are states
- Act are actions (for communication and synchronization)
- $\rightarrow \subseteq$ S x Act x S represents the evolution of the system
- I $\subseteq$ S is the set of initial states
- AP is a set of atomic propositions (encodes interesting properties of states)
- L : S $\rightarrow \mathcal{P}$(AP) is a labelling function associating states with the propositions they satisfy

State Based VS Action Based

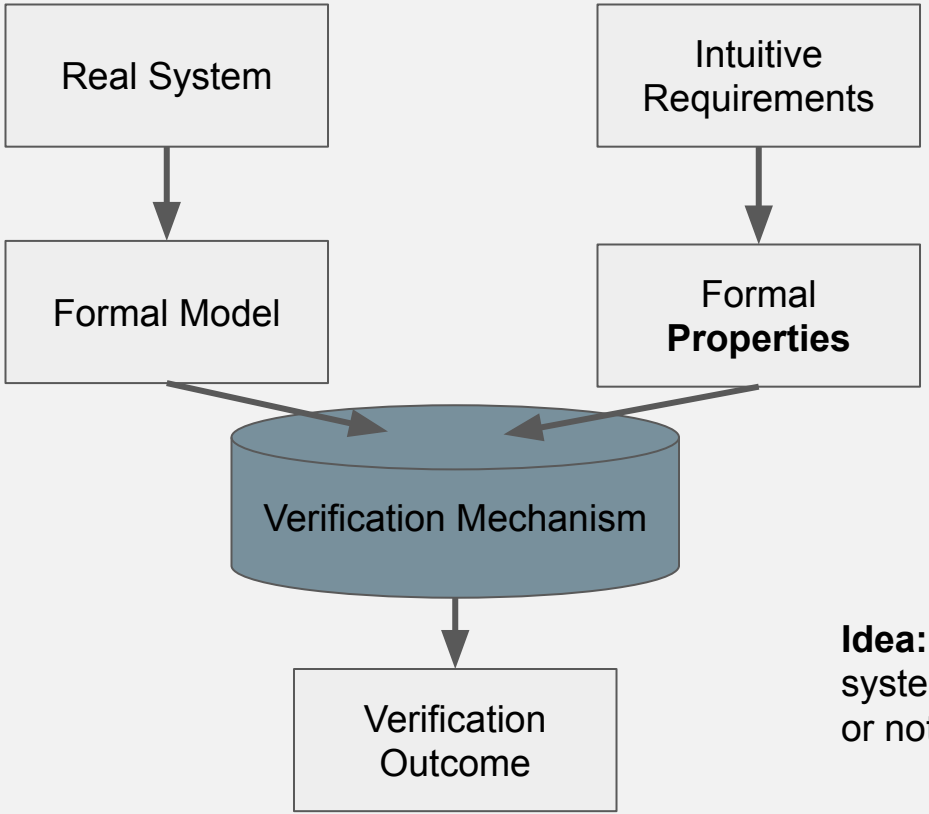
IMT SCUOLA ALTI STUDI LUCCA

Shared Variables	Message Passing
TS with states' internal variables	LTS with abstract states
We can (mostly) ignore actions	We can (mostly) ignore states
Focus is on the states	Focus is on the actions
AP speaks of variables states and other properties (e.g. being in a critical section)	ACT speaks of messages

State Based Verification

IMT
SCUOLA
ALTI STUDI
LUCCA

Property Verification



Idea: We will know if the system satisfies the properties or not!

A Linear Model: Trace Properties

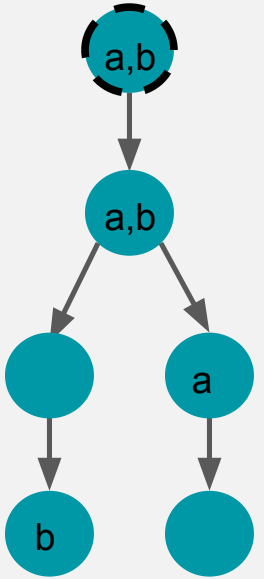
IMT SCUOLA ALTI STUDI LUCCA

Given a *Kripke Transition System* K = (S, Act, $\rightarrow$, I, AP, L)

Execution: $S_0, \mu_1, S_1, \mu_2, \ldots \mu_n, S_n$
such that $(S_{i-1}, \mu_i, S_i) \in \rightarrow$ and $S_0 \in I$
Path: $S_0, S_1, \ldots, S_n$
Trace: $L(S_0), L(S_1), \ldots, L(S_n)$

We assume Traces(K) are the maximal traces only
If t is a trace, we write t[i] for its ith set of atomic propositions

Traces of the example:
{a,b}, {a, b}, ø, {b}
{a,b}, {a, b}, {a}, ø



Notation: we omit the actions labeling the arrows, since they are irrelevant in this approach

A Linear Model: Trace Properties

IMT SCUOLA ALTI STUDI LUCCA

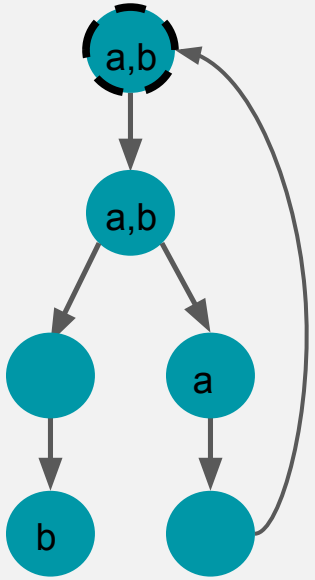
Notice: traces may be infinite!

Traces of the example:
({a,b}, {a, b}, {a}, ø)^n {a,b}, {a, b}, ø, {b}
({a,b}, {a, b}, {a}, ø)□

A **linear time property** is a set of (acceptable) traces

Satisfaction Relation: K ⊨ E iff Traces(K) ⊆ E

Example: the system in figure
- satisfies {t | exists i s.t. t[i] = {a,b} }?
- satisfies {t | exists i s.t. t[i] = {a} and t[i+1] = {a,b} }?
- satisfies {t | exists i s.t. t[i] = {a} }?
- satisfies {t | exists i s.t. t[i] = {b} }?



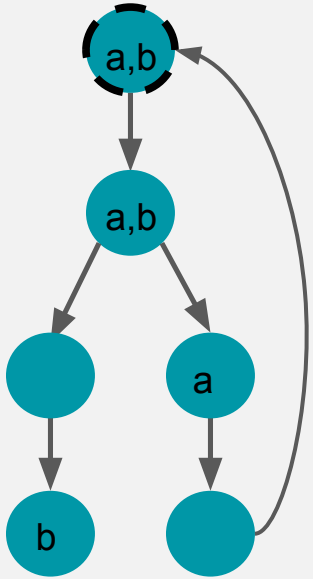
Finite VS Infinite Traces

IMT SCUOLA ALTI STUDI LUCCA

They are not the same!

Let E = { t | exists i s.t. t[i] = {b} }
E is satisfied by the figure if we only consider maximal finite traces

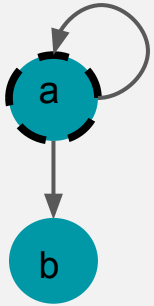
It is not satisfied otherwise!

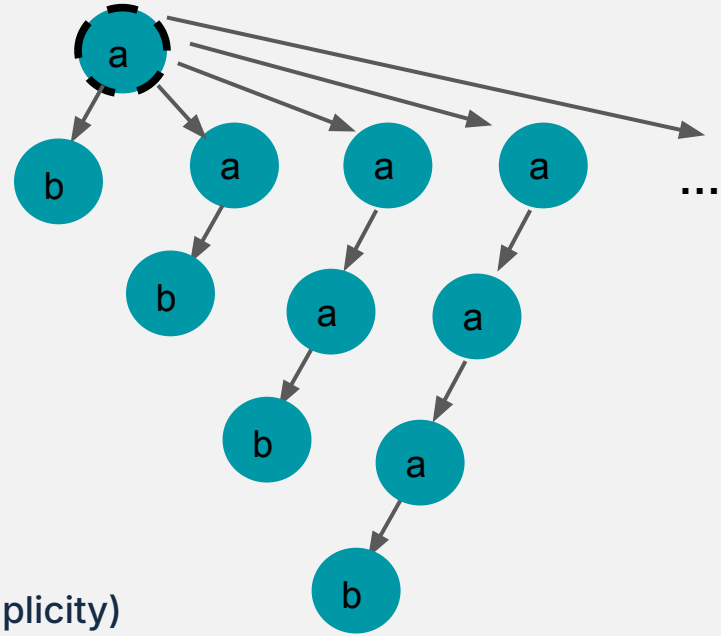


Finite VS Infinite Traces

IMT SCUOLA ALTI STUDI LUCCA

Note: the following KTSs have the same finite traces but not the same infinite ones





We will use infinite traces,
and assume KTSs to have no terminal state (for simplicity)

Refinement Preserve Trace Properties

IMT SCUOLA ALTI STUDI LUCCA

Theorem:

If the new system has strictly less traces, then the trace properties still holds

Example: when resolving non-determinism (e.g. fixing a scheduling policy)

Usage:

- if you refine your system, then you don’t need to verify it again
- you can verify a system for which you don’t know some detail (use non-determinism while modeling)

Trace Properties: Safety and Liveness

IMT SCUOLA ALTI STUDI LUCCA

Safety properties: *nothing bad will happen*
e.g.
- mutual exclusion
- deadlock freedom
- “every red phase is preceded by a yellow phase”

Liveness properties: *something good will happen*
e.g.
- “each waiting process will eventually enter its critical section”
- “each process will terminates at some point”

Safety Properties

IMT SCUOLA ALTI STUDI LUCCA

Safety properties: “no bad prefixes”

t ∉ E, then there is a bad prefix t′ of t such that every continuation of t′ is not in E

Invariants are the simplest safety properties:

An invariant is fully defined by a set of “bad” states

Bad ⊆ $\mathcal{P}(\mathcal{P}(\text{AP}))$

E = { t | ∀i . t[i] ∉ Bad }

The classical proposition encoding $\mathcal{P}(\mathcal{P}(\text{AP}))$ \ Bad is called the “invariant”

Invariants: “no bad states”

Invariant (and Safety) Example: Mutual Exclusion

IMT SCUOLA ALTI STUDI LUCCA

Recall: two processes cannot be in their critical section at the same time

Atomic Propositions AP = {wait1, wait2, crit1, crit2}

Bad = {{crit1, crit2}}

E = { t | $\forall$ i. crit1 $\notin$ t[i] or crit2 $\notin$ t[i] }

Liveness Properties

IMT SCUOLA ALTI STUDI LUCCA

Liveness properties: “something good will happen”

Each finite t can be extended to a t′ that is in E

Idea: the good thing can always happens later

Liveness Example: Starvation Freedom

IMT SCUOLA ALTI STUDI LUCCA

Recall: a process is starving if it waits indefinitely

Atomic Propositions AP = {wait1, wait2, crit1, crit2}

$$E = \{ t \mid \forall i.\ wait1 \in t[i] \text{ implies } \exists j \geq i\ .\ crit1 \in t[j] \text{ and } \forall i.\ wait2 \in t[i] \Rightarrow \exists j \geq i\ .\ crit2 \in t[j] \}$$

Safety and Liveness

IMT SCUOLA ALTI STUDI LUCCA

Two Results about Linear Time Properties

Disjointness: The only linear property that is both a safety and a liveness property is $\mathcal{P}(AP)$.

Coverage: For any linear property E there exists a safety property Esafe and a liveness property Elive such that

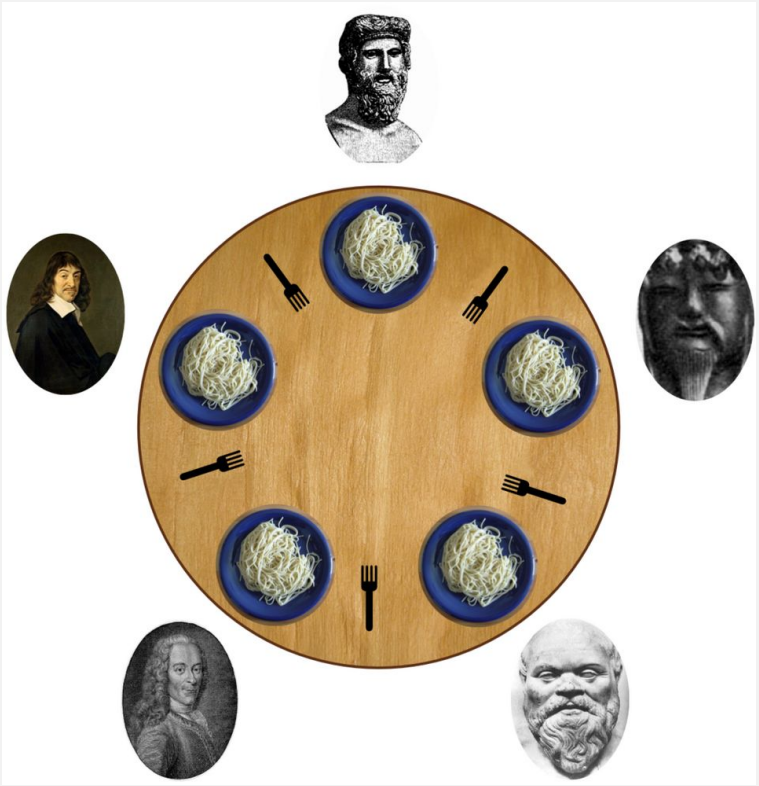
$$E = Esafe \cap Elive$$

Types of Trace Properties: Dining Philosophers Example

IMT SCUOLA ALTI STUDI LUCCA

Which property type?

- Each philosopher thinks infinitely often
- Two philosophers next to each other never eat at the same time
- Whenever a philosopher eats then he has been thinking at some time before



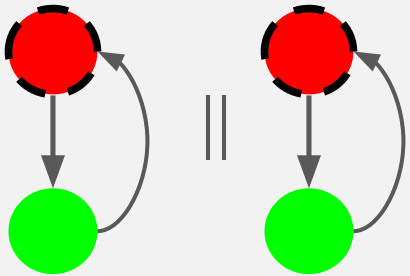
Liveness Modulo Fairness

IMT SCUOLA ALTI STUDI LUCCA

Observation: liveness properties are often “artificially” violated

Here “artificially” means “in our formal model but not in the real world”

Example: Two semaphores goes from green to red in parallel, is it true that each will become green infinitely often?



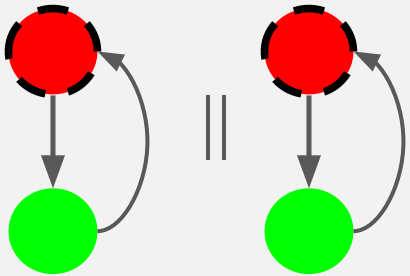
Liveness Modulo Fairness

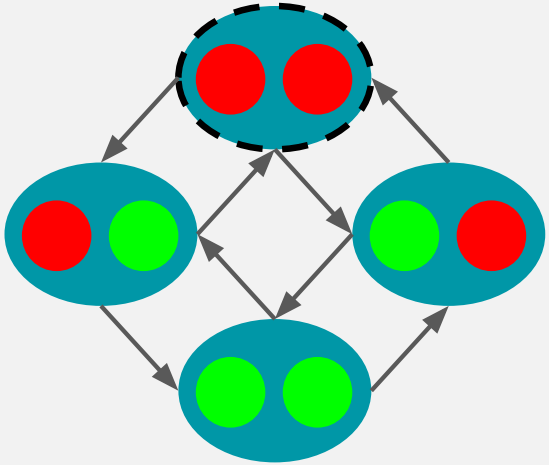
IMT SCUOLA ALTI STUDI LUCCA

Observation: liveness properties are often “artificially” violated

Here “artificially” means “in our formal model but not in the real world”

Example: Two semaphores goes from green to red in parallel, is it true that each will become green infinitely often?





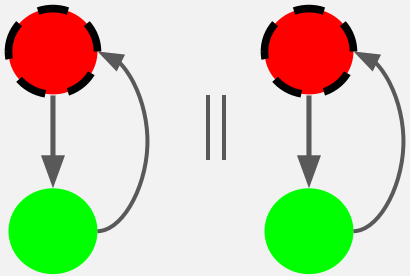
Liveness Modulo Fairness

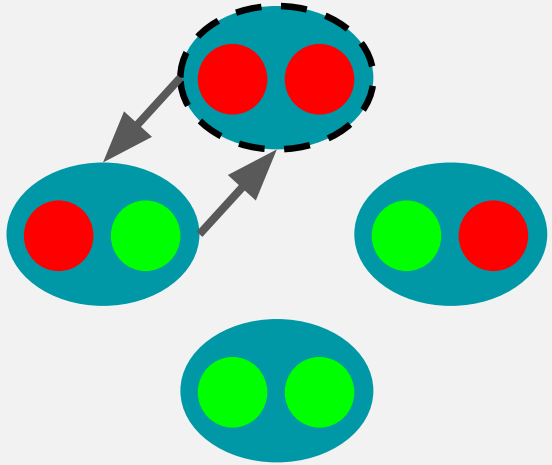
IMT SCUOLA ALTI STUDI LUCCA

Observation: liveness properties are often “artificially” violated

Here “artificially” means “in our formal model but not in the real world”

Example: Two semaphores goes from green to red in parallel, is it true that each will become green infinitely often?





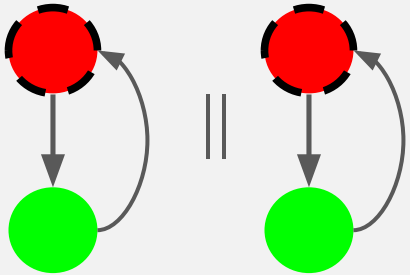
Liveness Modulo Fairness

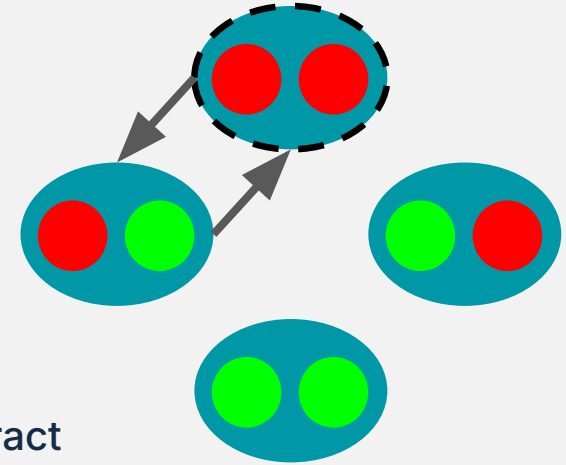
IMT SCUOLA ALTI STUDI LUCCA

Observation: liveness properties are often “artificially” violated

Here “artificially” means “in our formal model but not in the real world”

Example: Two semaphores goes from green to red in parallel, is it true that each will become green infinitely often?





Problem: interleaving is completely time-abstract

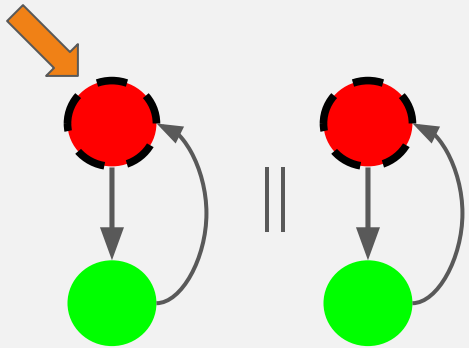
Liveness Modulo Fairness

IMT SCUOLA ALTI STUDI LUCCA

Notice: it is not true that K $\models$ “the first semaphore become green infinitely often”

BUT it is true that K1 $\models$ “the first semaphore become green infinitely often”

Where K1 is the KTS of just the first semaphore



Solution: Liveness Modulo Fairness

IMT SCUOLA ALTI STUDI LUCCA

Fairness allows to express known properties of the model: an appropriate resolution of the nondeterminism resulting from interleaving and competitions

Satisfaction Relation: $K \models_f E$ iff **FairTraces**$(K) \subseteq E$

(often fairness is defined using actions of the KTS, but states can be used to)

Nuances of Fairness

IMT SCUOLA ALTI STUDI LUCCA

Unconditional fairness: every process gets its turn infinitely often.

Strong fairness: every process that is enabled infinitely often gets its turn infinitely often.

Weak fairness: every process that is continuously enabled from a certain time on, gets its turn infinitely often

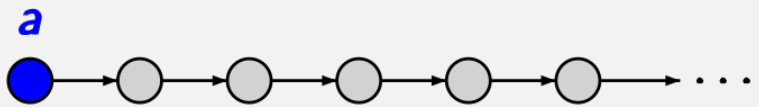
Linear Temporal Logic

IMT SCUOLA ALTI STUDI LUCCA

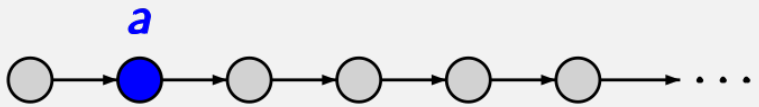
$$\varphi ::= \mathit{true} \mid a \mid \varphi_1 \wedge \varphi_2 \mid \neg\varphi \mid \bigcirc \varphi \mid \varphi_1 \,\mathsf{U}\, \varphi_2$$

where $a \in AP$ $\qquad$ $\bigcirc \mathrel{\widehat{=}}$ next $\qquad$ $\mathsf{U} \mathrel{\widehat{=}}$ until

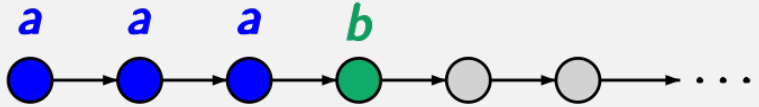
atomic proposition $a \in AP$



next operator $\bigcirc a$



until operator $a \,\mathsf{U}\, b$



Linear Temporal Logic

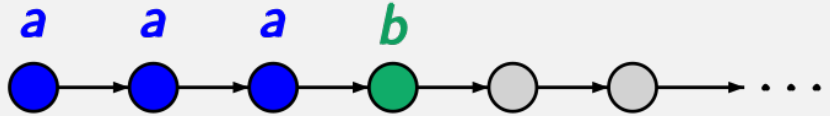
IMT SCUOLA ALTI STUDI LUCCA

derived operators:

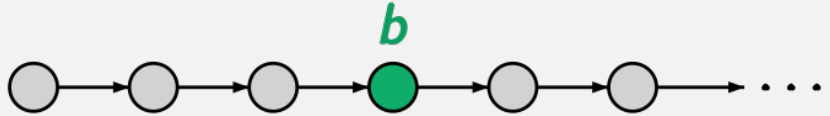
$\vee, \rightarrow, \ldots$ as usual

$\Diamond\varphi$	$\stackrel{\text{def}}{=}$	$\mathit{true}\ \mathsf{U}\ \varphi$	eventually
$\Box\varphi$	$\stackrel{\text{def}}{=}$	$\neg\Diamond\neg\varphi$	always

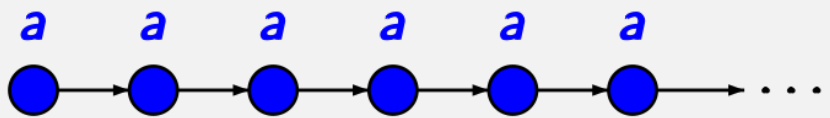
until operator
$a\ \mathsf{U}\ b$



eventually
$\Diamond b$



always
$\Box a$



for $\sigma = A_0\, A_1\, A_2\, \ldots \in \left(2^{AP}\right)^{\omega}$:

$$\sigma \models \mathit{true}$$

$$\sigma \models a \quad \text{iff} \quad A_0 \models a \text{ ,i.e., } a \in A_0$$

$$\sigma \models \varphi_1 \wedge \varphi_2 \quad \text{iff} \quad \sigma \models \varphi_1 \text{ and } \sigma \models \varphi_2$$

$$\sigma \models \neg\varphi \quad \text{iff} \quad \sigma \not\models \varphi$$

$$\sigma \models \bigcirc\varphi \quad \text{iff} \quad \mathit{suffix}(\sigma, 1) = A_1\, A_2\, A_3\, \ldots \models \varphi$$

$$\sigma \models \varphi_1\, \mathsf{U}\, \varphi_2 \quad \text{iff} \quad \text{there exists } j \geq 0 \text{ such that}$$

$$\mathit{suffix}(\sigma, j) = A_j\, A_{j+1}\, A_{j+2}\, \ldots \models \varphi_2 \quad \text{and}$$

$$\mathit{suffix}(\sigma, i) = A_i\, A_{i+1}\, A_{i+2}\, \ldots \models \varphi_1 \quad \text{for } 0 \leq i < j$$

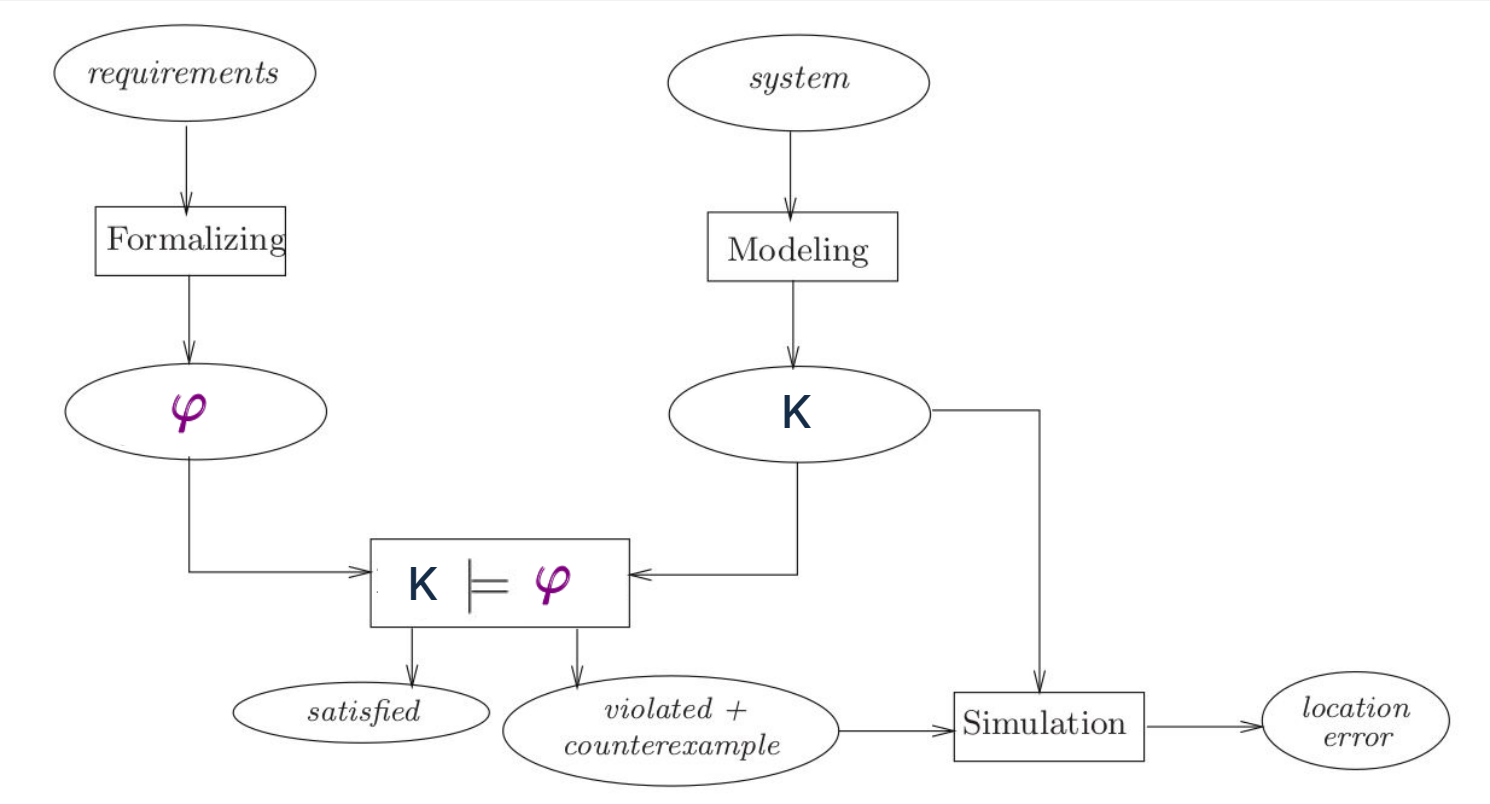
IMT SCUOLA ALTI STUDI LUCCA

Examples for LTL formulas:

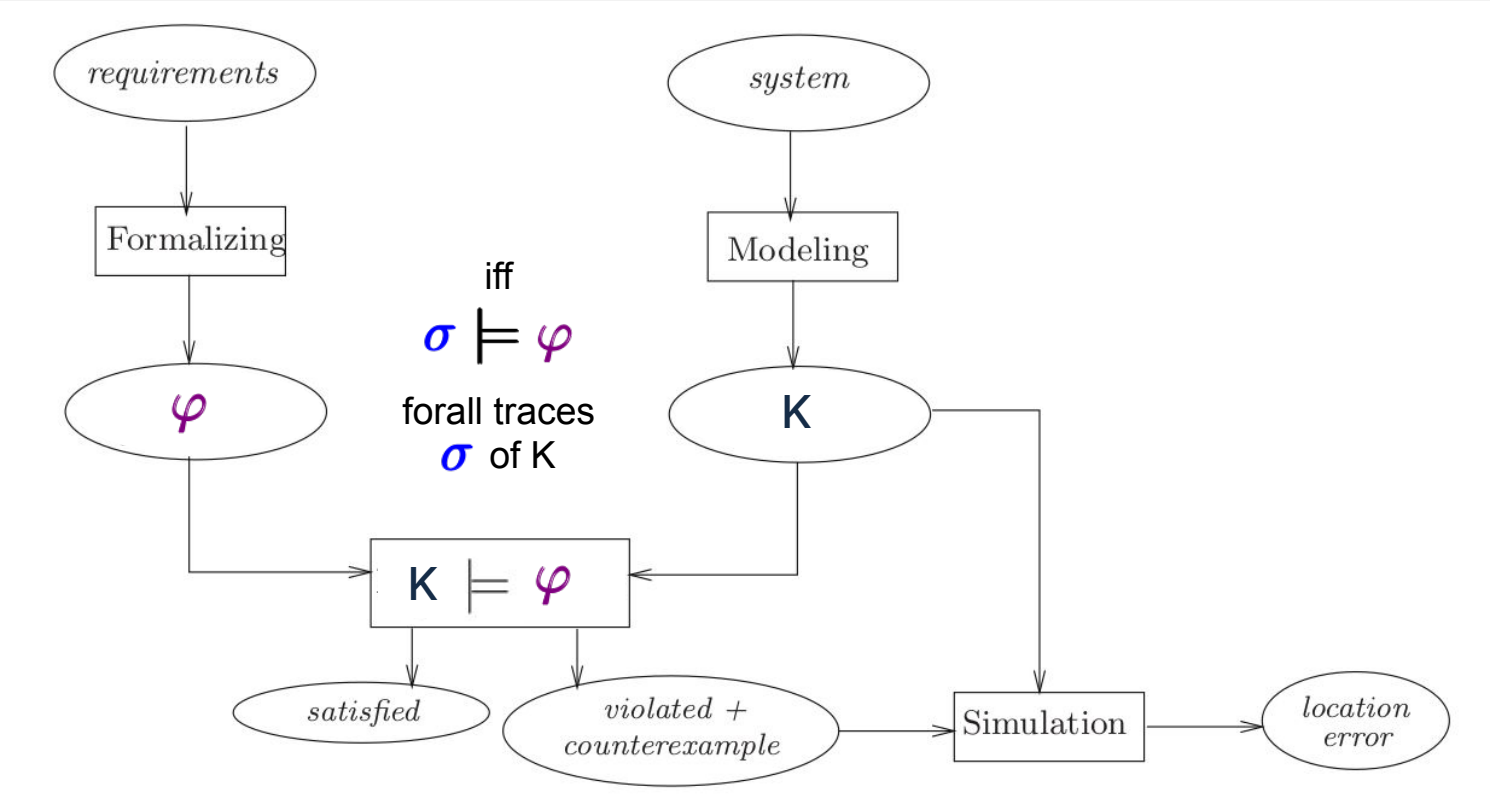
mutual exclusion:	$\Box(\neg \mathit{crit}_1 \vee \neg \mathit{crit}_2)$
railroad-crossing:	$\Box(\mathit{train\_is\_near} \rightarrow \mathit{gate\_is\_closed})$
progress property:	$\Box(\mathit{request} \rightarrow \Diamond \mathit{response})$
traffic light:	$\Box(\mathit{yellow} \vee \bigcirc \neg \mathit{red})$
infinitely often:	$\Box\Diamond \varphi$
eventually forever:	$\Diamond\Box \varphi$

Model Checking Linear Temporal Logic

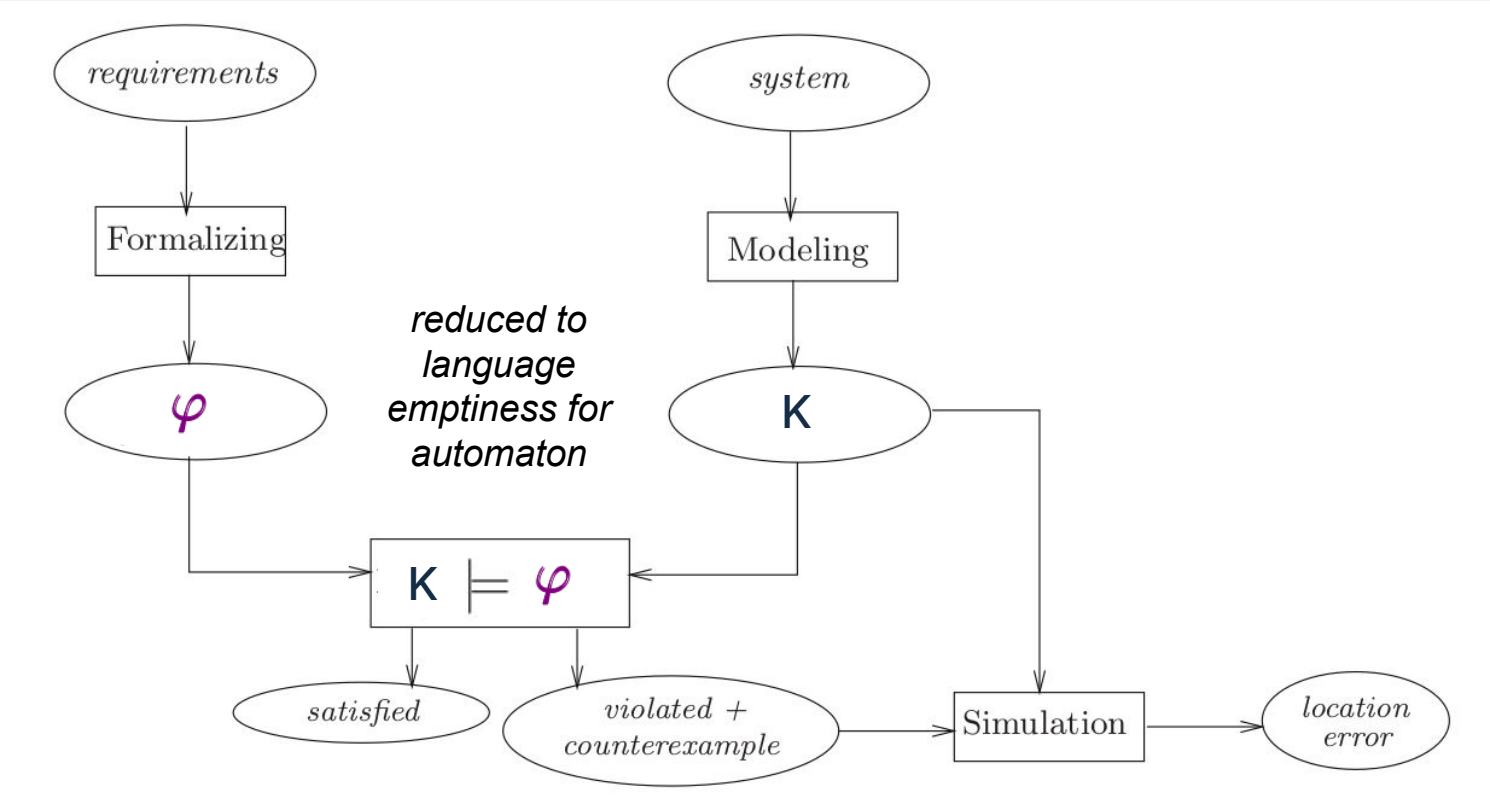
IMT SCUOLA ALTI STUDI LUCCA



Model Checking Linear Temporal Logic



Model Checking Linear Temporal Logic



Action Based Verification

IMT
SCUOLA
ALTI STUDI
LUCCA

Action Based Verification

IMT SCUOLA ALTI STUDI LUCCA

The general model is a **Kripke Transition System** : (S, Act, $\rightarrow$, I, AP, L)

We ignore states and only consider the branching structure and actions

The system under analysis is an **LTS** (S, Act, $\rightarrow$, I)

We focus on message passing!

For simplicity, we do not care of integer variables, only “constant messages” **a!** and **a?** representing abstract send and receive

Action Based Verification

IMT SCUOLA ALTI STUDI LUCCA

A CCS process
P ::= nil | τ.P | c!.P | c?.P | P + P | P ‖ P | P \ c

$$\frac{}{c!.P \xrightarrow{c!} P} \qquad \frac{}{c?.P \xrightarrow{c?} P}$$

$$\frac{P \xrightarrow{\mu} P'}{P + Q \xrightarrow{\mu} P'} \qquad \frac{}{\tau.P \xrightarrow{\tau} P} \qquad \frac{P \xrightarrow{\mu} P' \quad \mu \neq c!, c?}{P \setminus c \xrightarrow{\mu} P' \setminus c}$$

$$\frac{P \xrightarrow{\mu} P'}{P \parallel Q \xrightarrow{\mu} P' \parallel Q} \qquad \frac{P \xrightarrow{c!} P' \quad Q \xrightarrow{c?} Q'}{P \parallel Q \xrightarrow{\tau} P' \parallel Q'}$$

Behavioural Equivalence

IMT SCUOLA ALTI STUDI LUCCA

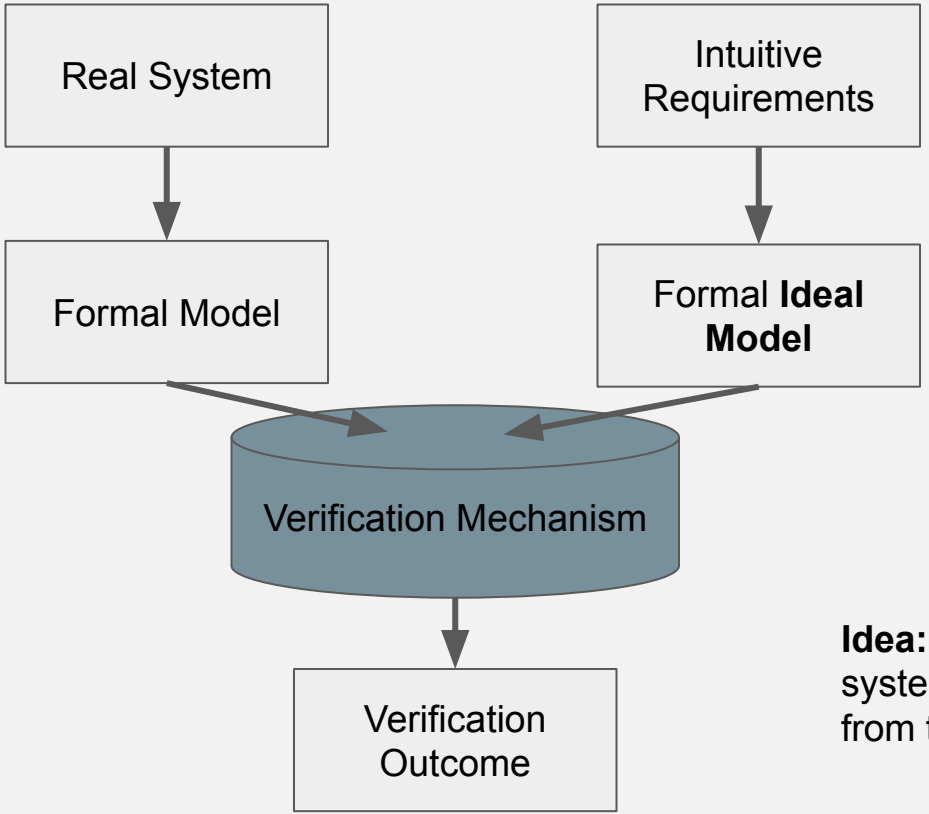
Two common approaches for system verification

- define a list of properties and check them
- define a desired specification and check equivalence with the system's model

Model Checking is of the first class

We will see an example of the second kind for action based verification

Behavioural Equivalence Verification



Idea: We will know if the real system is indistinguishable from the ideal one!

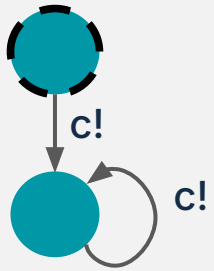
LTS are graphs, why not graph isomorphism?

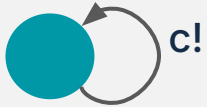
IMT SCUOLA ALTI STUDI LUCCA

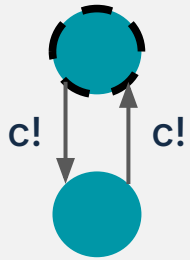
$(S, Act, \rightarrow, l) = (S', Act', \rightarrow', l')$ if a bijection $f : S \rightarrow S'$ exists such that
$f(l) = l'$
foreach s1, s2, μ, $s1 \xrightarrow{\mu} s2$ iff $f(s1) \xrightarrow{\mu}' f(s2)$

Because graph isomorphism is **too concrete** (distinguish too much)!

Intuitively, the following LTSs should be equivalent (they all send over c indefinitely)







Linear Time Semantics: Trace Equivalence

IMT SCUOLA ALTI STUDI LUCCA

(Finite) Trace Equivalence

Trace = Sequence of actions $\mu_1, \mu_2, \mu_3, \ldots, \mu_n$ such that

$$I \xrightarrow{\mu_1} s\_1 \xrightarrow{\mu_2} s\_2 \xrightarrow{\mu_3} \ldots \xrightarrow{\mu_n} s\_n$$

Notice: Traces of an LTS is closed for prefixes

if $t \in$ Traces(LTS) then prefixes(t) $\subseteq$ Traces(LTS)

...is this equivalence notion correct?

Black Box Approach to Distinguishability

IMT SCUOLA ALTI STUDI LUCCA

How to assess an equivalence?

With contexts, i.e. processes to be put in parallel: if a context can distinguish two processes deemed equivalent, then the equivalence is not good.

Intuitive meaning of “distinguishing context”

- Parallel composition (à la CCS)
- Termination in the context is immediately observable

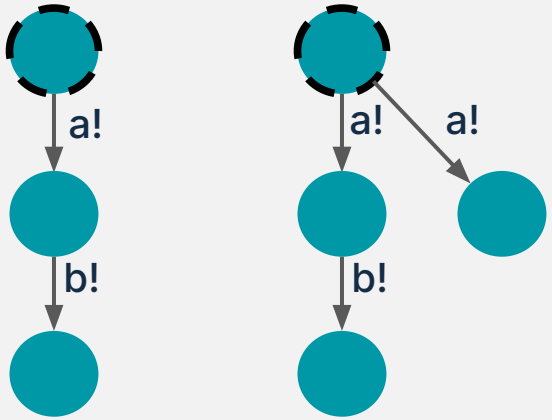
Linear Time Semantics: Trace Equivalence

IMT SCUOLA ALTI STUDI LUCCA

(Finite) Trace Equivalence does not work!

Consider the processes

a!.b!.nil VS a!.b!.nil + a!.nil
traces of both are {ϵ, a!, a!b!}



The process that does a?.b?.nil can **distinguish them**:
The one on the left it always terminate, while on the right it can wait indefinitely
(in our intuitive meaning of distinguishing context, termination should be visible).

Linear Time Semantics: Completed Trace Equivalence

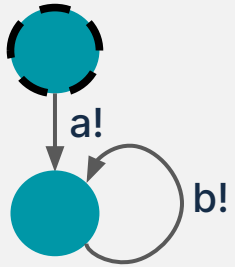
IMT SCUOLA ALTI STUDI LUCCA

(Finite) Completed Trace Equivalence

As before, but we only take the maximal traces

a!.b!.nil has maximal traces {a!b!}
a!.b!.nil + a!.nil has maximal traces {a!, a!b!}

(for the moment, ignore cases in which there are only infinite maximal traces)



Linear Time Semantics: Maximal Trace Equivalence

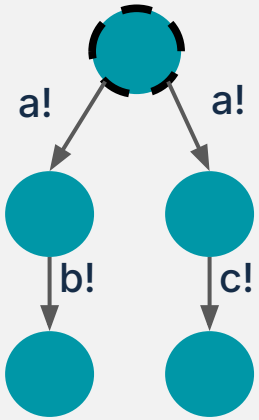
IMT SCUOLA ALTI STUDI LUCCA

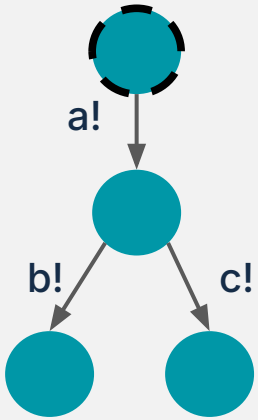
(Finite) Completed Trace Equivalence still has problems

a!.b!.nil + a!.c!.nil
has maximal traces {a!b!, a!c!}

a!.(b!.nil + c!.nil)
has maximal traces {a!b!, a!c!}

Still problems: The process a?.b?.nil can distinguish them.
On the right it always terminates, while on the left it may wait indefinitely





Linear Time – Branching Time Spectrum

IMT SCUOLA ALTI STUDI LUCCA

- The difference between a!.b!.nil + a!.nil is the **presence of a non-deterministic choice** (branching)
- The difference between a!.b!.nil + a!.c!.nil and a!.(b!.nil + c!.nil) is the **position of the non-deterministic choice** (branching)

This it the idea of a Linear Time – Branching Time Spectrum for Equivalences:

- Equivalence can ignore or look into the structure of the LTS
- Trace Equivalence is the most coarse grained
- Graph isomorphism is the most fine grained

Linear Time VS Branching Time Models

IMT SCUOLA ALTI STUDI LUCCA

	Trace Equivalence	Completed Trace Equivalence	…	Graph Isomorphism
a!.nil = a!.(c!.nil / c) + a!.nil	ok	ok		fail
a!.b!.nil ≠ a!.b!.nil + a!.nil	fail	ok		ok
a!.b!.nil + a!.c!.nil ≠ a!.(b!.nil + c!.nil)	fail	fail		ok

Linear Time VS Branching Time Models

IMT SCUOLA ALTI STUDI LUCCA

	Trace Equivalence	Completed Trace Equivalence	Bisimilarity	Graph Isomorphism
a!.nil = a!.(c!.nil / c) + a!.nil	ok	ok	ok	fail
a!.b!.nil ≠ a!.b!.nil + a!.nil	fail	ok	ok	ok
a!.b!.nil + a!.c!.nil ≠ a!.(b!.nil + c!.nil)	fail	fail	ok	ok

(Strong) Bisimilarity

IMT SCUOLA ALTI STUDI LUCCA

A relation R between states is a **bisimulation** if forall s1, s2. s1 R s2 implies

- forall μ, s1′. if $s1 \xrightarrow{\mu} s1'$ then exists s2′ s.t. $s2 \xrightarrow{\mu} s2'$ and s1′ R s2′
- forall μ, s2′. if $s2 \xrightarrow{\mu} s2'$ then exists s1′ s.t. $s1 \xrightarrow{\mu} s1'$ and s1′ R s2′

Bisimilarity is the union of all bisimulations.

Lemma: Bisimilarity is the largest bisimulation

Bisimilarity Game

IMT SCUOLA ALTI STUDI LUCCA

Idea: Alice wants to prove that P and Q are bisimilar, Bob wants to disprove it.
1) Bob choses a move from P to P′
2) Alice must find a move from Q to Q′ with the same label, if she cannot, she loses, otherwise the game restarts with P′ and Q′

P and Q are bisimilar iff Bob cannot find a winning strategy

Making Distinguishing Contexts More Formal

IMT SCUOLA ALTI STUDI LUCCA

Atomic observable: P↓ means that P can perform some visible action

Idea: reduction closure and barb equivalence are an observation!

A relation R is a barbed simulation if forall s1, s2. s1 R s2 implies

- s1↓ if and only if s2↓
- forall s1′, if $s1 \xrightarrow{\tau} s1'$ then exists s2′ s.t. $s2 \xrightarrow{\tau} s2'$ and s1′ R s2′
- forall s2′, if $s2 \xrightarrow{\tau} s2'$ then exists s1′ s.t. $s1 \xrightarrow{\tau} s1'$ and s1′ R s2′

Barbed Bisimilarity is the union of all barbed bisimulations.

Making Distinguishing Contexts More Formal

IMT SCUOLA ALTI STUDI LUCCA

Final Step: Quantify over all possible observers!

Recall: LTSs can be composed in parallel

Given s1 from the LTS T1 and s2 from the LTS T2,
we write s || t for the state s || t in the LTS T1 || T2.

Two states s1, s2 are **Barbed Congruent** if s1 || s is barbed bisimilar to s2 || s for every state s

Theorem: Barbed congruence coincides with bisimilarity!

Strong VS Weak Models

IMT SCUOLA ALTI STUDI LUCCA

In reality, we often want to ignore the number of tau-transitions

A relation that consider the number of taus is called **Strong**
One that ignores it is called **Weak**

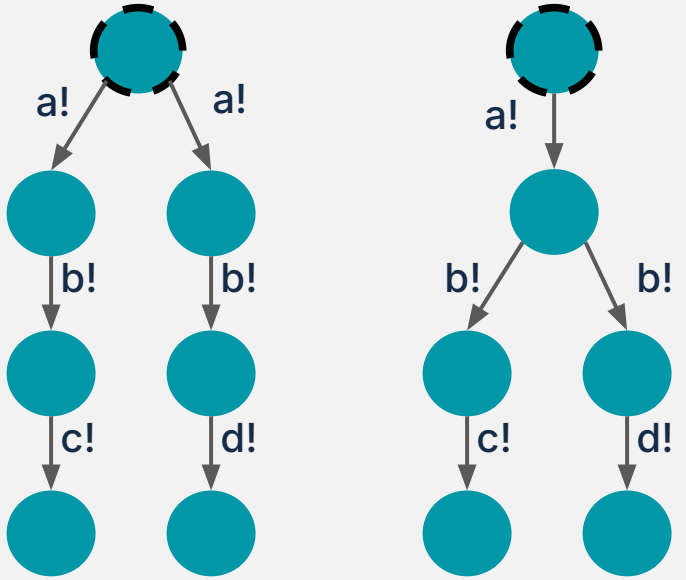
Usually strong equivalences imply weak ones

Some weak bisimilarities exists... strong here only for simplicity

Extra: Testing Equivalence

IMT SCUOLA ALTI STUDI LUCCA

The Apple of Discord: Should we identify these?



Notice: still a problem of branching structure.

The two systems branch into paths that cannot give either c! or d! in the end, but they branch at different points in time.

The point is: Can I discover where the branch happens?

Sources: Books

IMT SCUOLA ALTI STUDI LUCCA

Robin Milner, Communication and Concurrency
> Introduction of bisimilarity and CCS

C. A. R. Hoare, Communicating Sequential Processes
> Introduction of CSP

A.W. Roscoe, The Theory and Practice of Concurrency
> Updated CSP

Roberto Bruni and Ugo Montanari, Models of Computation: Part IV Concurrent Systems
> You can find an introduction to formal models for concurrency and behavioural equivalences

Christel Baier and Joost-Pieter Katoen, Principles of Model Checking
> A good guide to model checking: traces, omega-traces, CTL, LTL, etc

Bart Jacobs, Introduction to Coalgebra: Towards Mathematics of States and Observation
> A Nice Introduction to Coalgebras

Sources: Papers

IMT SCUOLA ALTI STUDI LUCCA

Filippo Bonchi, Fabio Gadducci, Giacoma Valentina Monreale, A General Theory of Barbs, Contexts, and Labels
> You can find something about the relation between contextual and labelled equivalence

Robin Milner, Davide Sangiorgi: Barbed Bisimulation
> Main work about contextual equivalence and bisimilarity

Peter Selinger. First-Order Axioms for Asynchrony.
> Sellinger Axioms for Asynchrony

R. De Nicola, M.C.B. Hennessy, Testing equivalences for processes
> Testing preorder initial definition

R.J. van Glabbeek, The Linear Time - Branching Time Spectrum (I and II)
> Compare different semantics in terms of their branching/linearity

Scuola IMT Alti Studi Lucca

Sede Legale
Piazza San Ponziano, 6
55100 Lucca, LU

Campus
Piazza San Francesco, 19
55100 Lucca, LU

Via Brunero Paoli, 31
55100 Lucca, LU

Telefono
+39 0583 4326561

Email
info@imtlucca.it

IMT
SCUOLA
ALTI STUDI
LUCCA

imtlucca.it

IMT School for Advanced Studies Lucca

IMT
SCUOLA
ALTI STUDI
LUCCA

Legal headquarters
Piazza San Ponziano, 6
55100 Lucca (Italy)

Campus
Piazza San Francesco, 19
55100 Lucca (Italy)

Via Brunero Paoli, 31
55100 Lucca (Italy)

Telephone
+39 0583 4326561

Email
info@imtlucca.it

imtlucca.it